

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №6

Выполнил:

Шалунов Андрей

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить rabbitMQ/kafka;

Реализовать межсервисное взаимодействие посредством rabbitMQ/kafka.

1. Подключение RabbitMQ

В проект был добавлен RabbitMQ как отдельный контейнер в docker-compose.yml. Для удобства проверки был включен management интерфейс, чтобы можно было смотреть состояние обменников, очередей и наличие сообщений.

2. Настройка переменных окружения

В сервисы, которые работают с RabbitMQ, была добавлена переменная окружения AMQP_URL=amqp://rabbitmq:5672. После добавления переменной сервисы получили возможность устанавливать соединение с RabbitMQ без ручной настройки адресов.

3. События пользователей

Для синхронизации данных о пользователях между сервисами был реализован обмен событиями через RabbitMQ. В user-service добавлена публикация событий при создании и обновлении пользователя. Для этого создан exchange user.events типа fanout, чтобы одно событие доставлялось всем подписчикам. При регистрации или обновлении профиля user-service отправляет сообщение с типом события и актуальными данными пользователя.

В booking-service и message-service были добавлены consumer'ы, которые подписываются на user.events через собственные очереди. Каждая очередь привязывается к exchange, и при получении события сервис обновляет локальную таблицу User в своей базе.

4. Сценарий “Бронь”

Для демонстрации межсервисного взаимодействия был реализован сценарий “Бронь” через события. В booking-service после создания брони в базе публикуется событие booking.created в exchange booking.events типа topic.

property-service подписывается на событие booking.created через очередь booking.events.property. После получения события сервис проверяет, существует ли объект и доступен ли он для бронирования. Если объект доступен, property-service меняет статус объекта на reserved и публикует событие property.reserved. Если объект не найден или недоступен, публикуется событие booking.rejected с причиной отказа.

5. Проверка работоспособности

Проверка выполнялась через Postman и через RabbitMQ UI. Для проверки событий пользователя выполнялось обновление профиля, например PATCH /api/users/:id, после чего в логах booking-service и message-service было видно, что consumer получил событие и обновил локального пользователя. Для проверки сценария “Бронь” создавалась бронь через POST /api/bookings/, затем проверялось, что в property-service изменился статус объекта на reserved

6. Вывод

RabbitMQ был подключен и настроен, а в проекте реализовано межсервисное взаимодействие через события. В результате появились два рабочих сценария: синхронизация пользовательских данных через user.events и бизнес цепочка “Бронь” через booking.events, где сервисы реагируют на события асинхронно и не зависят от прямых HTTP-вызовов друг к другу.